

Error . Flaw . Imperfection . Must Sterilize



aka Elegant Error Handling with D

by Walter Bright
<https://x.com/WalterBright>
Dconf Sep 2026

Having to deal with diagnosing errors, reporting them, and recovering from them is not the funnest part of programming. Too often they are like a fly stuck on a freshly painted masterpiece. I will be showing many ways to deal with errors. Some I do not care for, and some are very elegant. You might even find enjoyment in the challenge of making error handling a work of art!

Categories of Errors

- Programming Bugs
- Out of Resources
- Environmental Errors (i.e. bad input)

Programming Bugs

- When detected, print context information and exit program
- Do not attempt to continue operating

Out of Resources

- Out of memory
- Out of stack space

Nothing to do but log
message and exit

Environmental Errors (Bad Input)

- Unexpected input
- Running out of disk space
- Network failure
- Bad values in things like environment and system settings

C Style Error Codes

```
error = printf("hello ");  
if (error)  
    failed();  
error = printf("betty\n");  
if (error)  
    failed();
```

Does Anyone Do That?

And So The Error Goes
Undetected

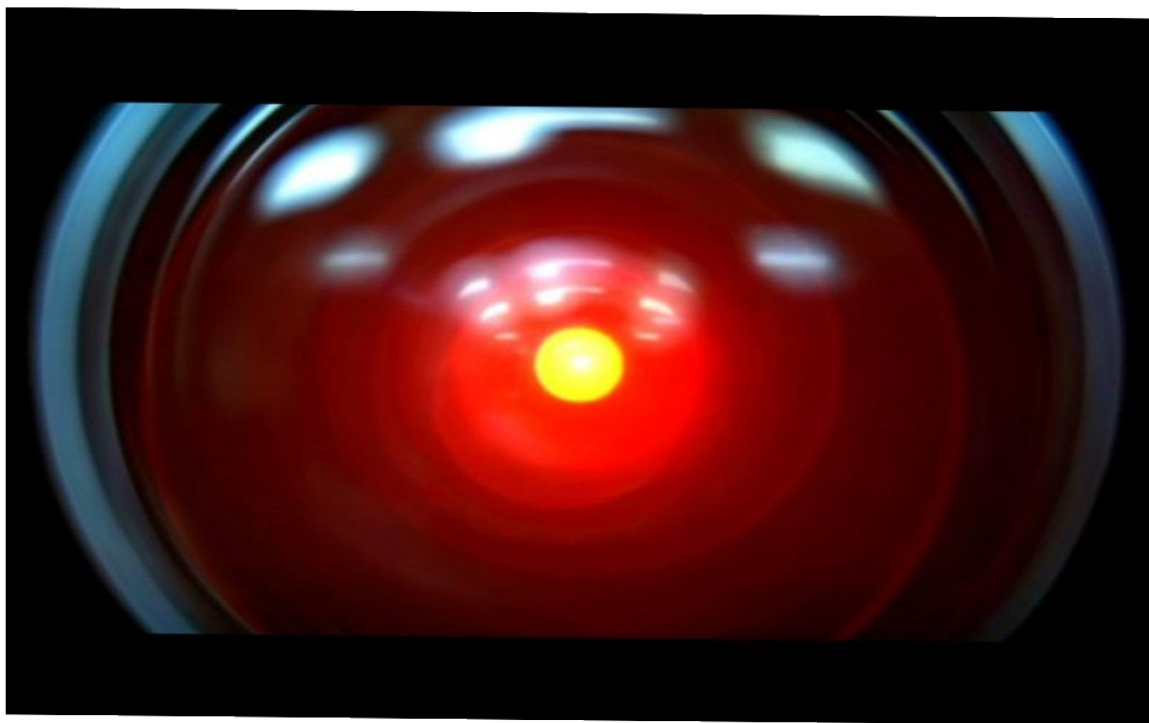


photo from 2001: A Space Odyssey

“It can only be attributable to human error.”

– Hal 9000

Sticky Error

```
int adds()(int x, int y, ref bool overflow) {  
    long r = cast(long)x + cast(long)y;  
    if (r < int.min || r > int.max)  
        overflow = true;  
    return cast(int)r;  
}
```

```
int muls()(int x, int y, ref bool overflow) {  
    long r = cast(long)x * cast(long)y;  
    if (r < int.min || r > int.max)  
        overflow = true;  
    return cast(int)r;  
}
```

```
bool overflow;  
int x = adds(a, b, overflow);  
int y = muls(x, 3, overflow);  
if (overflow)  
    printf("integer overflow\n");
```

Sticky Error for printf

The sticky error can be tested by calling `ferror()`, and then clearing it with `clearerror()`.

But, do you really want to be constrained by `stdout`? What if it's a system without `stdout`? `stdout` is a little too specific.

Sink Delegate instead of stdout

```
void delegate (ubyte) nothrow sink;
```

And use it like:

```
char[40] buf = void;  
size_t i;  
foreach(dchar d; "a\n\r\t\b\f\0\x11\u7233\u00017233"d) {  
    void sink(ubyte c) { buf[i++] = c; }  
    writeCharLiteral(d, &sink);  
}
```

An Abstract Class Sink

```
abstract class ErrorSink {  
    nothrow extern (C++):  
  
        void verror(const(char)* format, va_list ap);  
  
        void error(const(char)* format, ...) {  
            va_list ap;  
            va_start(ap, format);  
            verror(format, ap);  
            va_end(ap);  
        }  
}
```

Ignore the Errors

```
class ErrorSinkNull : ErrorSink {  
    nothrow: extern (C++): override:  
  
        void verror(const(char)* format, va_list ap)  
        { }  
}
```

Sticky Errors for Sinks

```
class ErrorSinkLatch : ErrorSinkNull {  
    nothrow:  
    extern (C++):  
    override:  
  
    bool sawErrors;  
  
    void verror(const(char)* format, va_list ap) {  
        sawErrors = true;  
    }  
}
```


Send Errors to stderr

```
class ErrorSinkStderr : ErrorSink {  
    import core.stdc.stdio;  
    import core.stdc.stdarg;  
  
    nothrow: extern (C++): override:  
  
    void verror(const(char)* format, va_list ap) {  
        fputs("Error: ", stderr);  
        vfprintf(stderr, format, ap);  
        fputc('\n', stderr);  
    }  
}
```

Rich Error Type Result!(T, E)

```
struct Result(T, E) {  
    private union { T value_; E error_; }  
    bool isOk;  
    static Result!(T, E) ok(T val) {  
        Result!(T, E) res;  
        res.isOk = true;  
        res.value_ = val;  
        return res;  
    }  
    static Result!(T, E) err(E error) {  
        Result!(T, E) res;  
        res.isOk = false;  
        res.error_ = error;  
        return res;  
    }  
    T value() { assert(isOk); return value_; }  
    E error() { assert(!isOk); return error_; }  
}
```

```
enum FileError { NotFound, PermissionDenied }
```

```
Result!(string, FileError) readFile(string path) {  
    if (path == "secret.txt") {  
        return typeof(return).err(FileError.PermissionDenied);  
    }  
    return typeof(return).ok("File contents here!");  
}
```

```
void main() {  
    auto res = readFile("secret.txt");  
    if (!res.isOk) {  
        Switch (res.error) {  
            case FileError.NotFound: writeln("File not found."); break;  
            case FileError.PermissionDenied: writeln("Access denied."); break;  
        }  
    } else  
        writeln(res.value);  
}
```

I'm not a big fan of `Option` types for return values, as it means having to deal with the errors at every function return, like the `printf()` example earlier

Exception Handling

```
import std.stdio;

int mars(int x) {
    try {
        func(x);
    }
    catch (Exception e) {
        writeln("error %s", e.msg);
    }
    writeln("done");
    return 0;
}

void func(int x) {
    if (x != 3)
        throw new Exception("error");
}
```

Exception Handling Downsides

- Callers must be aware that calling a function means the function can throw an exception and unexpectedly unwind its state
- Can deal with this with destructors
 - But this can mean making Potemkin Village objects just to have a destructor
- Wind up with invisible, often untested, code paths

Transactions

Both `dofoo()` and `dobar()` must succeed, or the transaction has failed. If either failed, it must be restored to the state where neither happened.

```
Transaction transaction() {  
    Foo f = dofoo();  
    Bar b = dobar();  
  
    return Transaction(f, b);  
}
```

RAII Approach

```
struct FooX {  
    Foo f;  
    bool commit;  
    @disable this();  
  
    static FooX create() {  
        auto fx = FooX.init;  
        fx.f = dofoo();  
        return fx;  
    }  
  
    ~this() {  
        if (!commit)  
            dofoo_undo(f);  
    }  
}
```

```
Transaction transaction() {  
    auto fx = FooX.create();  
    Bar b = dobar();  
    fx.commit = true;  
    return Transaction(fx.f, b);  
}
```


Try-Catch Approach

```
Transaction transaction() {  
    Foo f = dofoo();  
    try {  
        Bar b = dobar();  
        return Transaction(f, b);  
    }  
    catch (Exception o) {  
        dofoo_undo(f);  
        throw o;  
    }  
}
```

scope(failure) Solution

```
Transaction transaction() {  
    Foo f = dofoo();  
    scope(failure) dofoo_undo(f);  
  
    Bar b = dobar();  
    return Transaction(f, b);  
}
```

Scaling It Up

```
Transaction transaction() {  
    Foo f = dofoo();  
    scope(failure) dofoo_undo(f);  
  
    Bar b = dobar();  
    scope(failure) dobar_unwind(b);  
  
    Def d = dodef();  
    return Transaction(f, b, d);  
}
```

Floating Point NaN

- `float f = 0.0; // initializes to 0.0`
- `float f; // initializes to NaN`
- `NaN op f // result is NaN`
- `f op NaN // result is NaN`
- `NaN == NaN // result is false`
- `NaN != NaN // result is true`

NaN Propagates an Error

- I dub that “poisoning” the results
- So errors are not overlooked
- So initializations are intentional, not accidentally set to 0
- Yes, `!=` and `==` do not propagate the error
 - But neither does 0.0, so NaN is not *worse* than default initializing to 0.0

Bad Unicode

- Not all code unit sequences are valid
- What to do when encountering them?
 - Most practical thing is just don't worry about it
- Unless converting between UTF-8, UTF-16, and UCS-32
 - Because there won't be a valid code point to convert to

Enter Autodecode Stage Left!

- If there's an invalid sequence, it throws an exception
 - Meaning memory allocation
 - Meaning the whole exception overhead
 - Burdening everything that uses Autodecode

Best Practice

- Convert bad sequences to Replacement Character
- Which is U+FFFD
- 

Parsing Errors

- Consider this invalid expression:
 - `int x = 4 + / 3;`
- I would code the parser to guess at what was intended
- The problem is it was always a bad guess, and the result was a cascade of ever-more incomprehensible attempts to guess what was actually meant
 - Which meant only the first error message was valid

ErrorExp

```
class ErrorExp : Expression {  
    this() {  
        super(Loc.initial, EXP.error);  
        type = Type.error;  
    }  
    static ErrorExp get () {  
        if (errorexp is null)  
            errorexp = new ErrorExp();  
        return errorexp;  
    }  
    __gshared ErrorExp errorexp;  
}
```

Any Expression Error Returns ErrorExp

- And if ErrorExp is an operand of any Expression, the Expression is replaced with ErrorExp
- This eliminated cascaded error messages
- And ErrorExp's themselves never cause an error message
-

More “Black Hole” Types

- `ErrorType`
- `ErrorStatement`
- `ErrorInitializer`

Define Errors Out Of Existence

```
string subString(string s, int start, int end)
{
    return string[start .. end];
}
```

Alternatively:

```
string subString(string s, int start, int end)
{
    if (start < 0) start = 0;
    if (end > s.length) end = s.length;
    return string[start .. end];
}
```

References

- A Philosophy of Software Design – Ousterhout
- <https://dlang.org/spec/statement.html#scope-guard-statement>
- <https://dlang.org/articles/exception-safe.html>